

Deploying a Three-Tier Application LIVE on AWS EKS

Watch the live session on YouTube (2h 35m):

youtube.com/watch?v=YIUa3t9Aaic

A Hands-On Project Guide — containerize a React + Node.js + MongoDB app, build an Amazon EKS cluster with eksctl, and expose it to the internet through an ALB Ingress.

What you'll do

Toolbox	One EC2 control machine with the AWS CLI, Docker, kubectl, eksctl & Helm
Images & ECR	Build each tier's image and push it to a private Amazon ECR registry
EKS cluster	Spin up a managed cluster with a single eksctl command
Three tiers	MongoDB (Secret + PV/PVC), a Node.js API, and a React frontend
ALB Ingress	One AWS load balancer routing / to the frontend and /api to the backend
Cleanup	Tear it all down with eksctl so nothing keeps billing

Compiled from the TrainWithShubham live session *Deploying a Three-tier Application LIVE on AWS EKS*. Written in plain language for beginners — every command is explained, and every manifest is taken straight from the project's GitHub repository so you can verify it yourself.

Table of Contents

Every entry is a live link — click it to jump straight to that topic.

Three-Tier Apps & the Plan	3
1. What a Three-Tier App Is	3
2. A Two-Minute Kubernetes Refresher	4
3. Meet the App and the Plan	4
4. The Tools You'll Need	5
Deploying LIVE on Amazon EKS	6
5. Setting Up the Toolbox	6
6. Containerizing the App and Pushing to ECR	7
7. Creating the EKS Cluster	8
8. Deploying the Three Tiers	9
9. Exposing the App with an ALB Ingress	15
10. Cleanup - Don't Skip This	18
Appendices	18
11. Command Cheat Sheet	18
12. Important Links and Repositories	20
13. Interview Questions and Answers	20
14. Glossary	21
15. Video Chapter Index	22

Three-Tier Apps & the Plan

What a three-tier app is, a fast Kubernetes refresher, the app you'll deploy, and the tools you need.

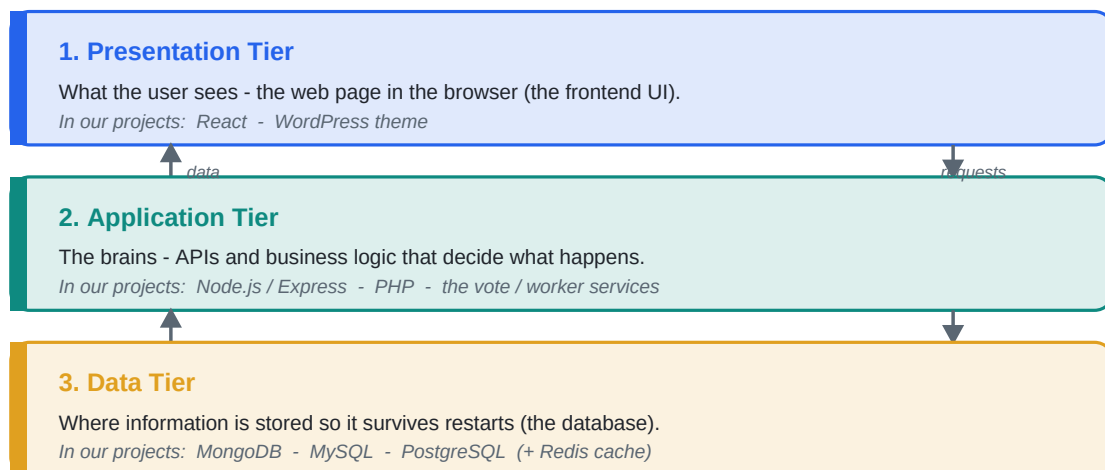
1. What a Three-Tier App Is

Watch from 0:02:14

This guide is hands-on. From start to finish you will take a real **three-tier to-do application** and deploy it **live on Amazon EKS** — a public URL, a real load balancer, the works. Before the first `kubectl apply`, though, let's nail down the one idea the whole project is built on: the **tier**.

A **tier** is a layer of an application that has one job and can be scaled, updated, and reasoned about on its own. Almost every web app you use is split into three tiers.

The three tiers of a modern web application



- The **Presentation tier** is what the user sees and clicks: the web page in the browser. It is often called the **frontend**. In this project it is a **React** single-page app.
- The **Application tier** is the brain: the code that receives requests, applies the business rules, and decides what happens. It is often called the **backend** or the **API**. Here it is a **Node.js / Express** server.
- The **Data tier** is the memory: the database where information is stored so it survives restarts. In this project that is **MongoDB**.

ANALOGY Think of a restaurant. The **dining room** (presentation) is where you sit and order. The **kitchen** (application) takes your order and cooks. The **pantry/fridge** (data) stores the ingredients. Each can be improved on its own — a bigger dining room doesn't require a new fridge.

Why split an app into tiers?

Because each tier has different needs, and separating them lets you handle each need independently:

- **Scale independently.** If lots of users are browsing but few are writing data, you add more frontend copies without touching the database.
- **Update independently.** You can ship a new frontend design without redeploying the backend.
- **Fail safely.** A crash in one tier shouldn't take the others down; Kubernetes can restart just the broken piece.
- **Secure the data.** The database can be locked away so *only* the backend can reach it — never the public internet.

Kubernetes is built for exactly this shape. Each tier becomes a **Deployment** (a set of identical Pods), each gets a **Service** (a stable address), and Kubernetes keeps them all alive and connected. That is the pattern you will build in this guide.

KEY IDEA A "three-tier app" is just **frontend + backend + database**, each running as its own set of Pods behind its own Service. Learn to wire those three together and you can deploy almost any real application.

2. A Two-Minute Kubernetes Refresher

This is a *project* guide, so we move fast and learn by doing. If a term below is brand new to you, that's fine — you'll see each one in action within a few pages. (For a slow, from-scratch tour of these same ideas, see the companion guide *Deploying Apps with Kubernetes*.)

Here is the whole vocabulary you need, in one table.

Term	In one line	Analogy
Cluster	A group of machines Kubernetes runs your apps across	A company with many workers
Node	One machine (a server / EC2 instance) in the cluster	One employee's desk
Pod	The smallest unit you deploy: a wrapper around one or more containers	A house for a container
Deployment	Keeps <i>N</i> identical Pods running, and heals/scales them	A manager who keeps <i>N</i> people staffed
Service	One stable address + load-balancing for a set of Pods	A department's front desk phone number
Namespace	A folder that isolates one app's objects from another's	A floor of the building for one team
Secret / ConfigMap	Store passwords / configuration outside your image	A locked drawer / a settings sheet
PV / PVC	Storage that outlives a Pod, and the request for it	A warehouse unit / the lease on it
Ingress	Routes outside web traffic (by host/path) into Services	The building's front reception & signage

Two ideas are worth burning into memory now, because this project leans on both:

- **Everything is declarative.** You write a YAML file describing the *desired state* ("I want 2 copies of this app"), hand it to the cluster with `kubectl apply`, and Kubernetes makes reality match — and keeps it matching. Those YAML files are called **manifests**.
- **Pods find each other by Service name, not IP.** A Pod's IP changes every time it restarts. A Service name (like `mongodb-svc`) never does. So the backend always reaches the database at `mongodb-svc`, no matter how many times the database Pod is recreated. This is **service discovery**, and it's how the tiers stay connected.

INTERVIEW What's the difference between a Deployment and a Service? A **Deployment** manages the Pods (how many copies, which image, healing, scaling). A **Service** gives those Pods one stable network address and load-balances across them. You almost always create them as a pair: a Deployment to run the app, a Service to reach it.

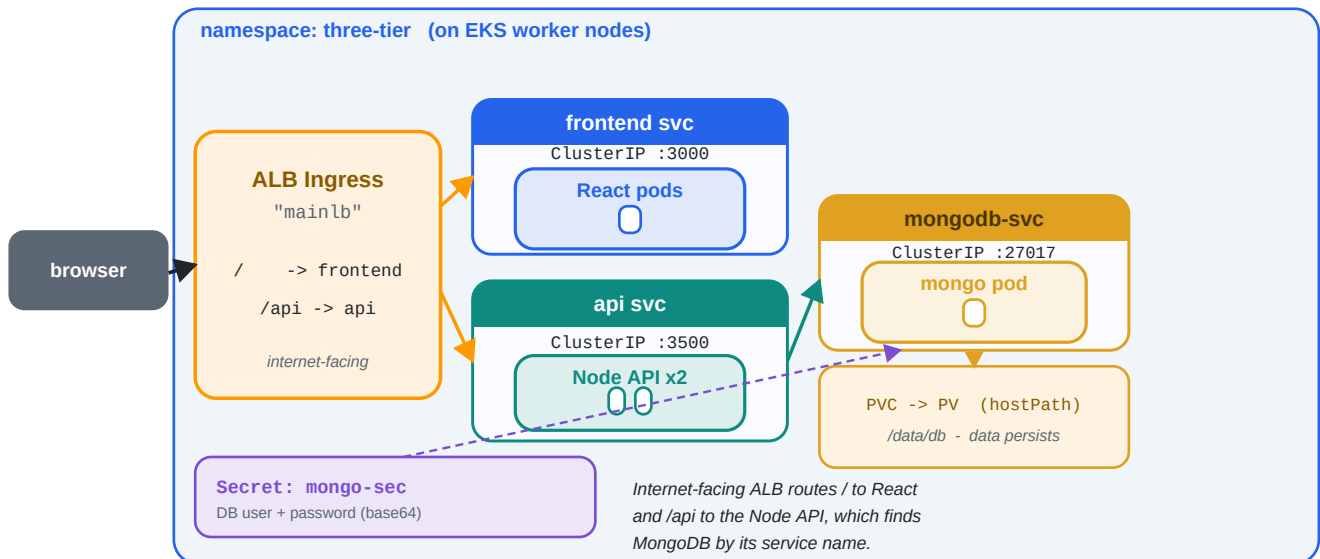
3. Meet the App and the Plan

Watch from 0:00:22

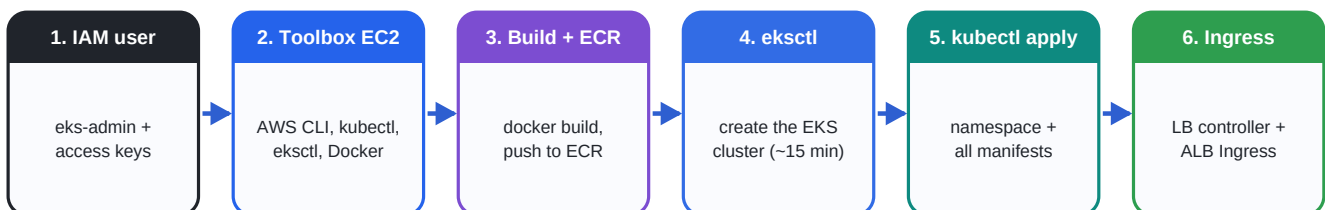
This project deploys a **three-tier to-do app** onto **Amazon EKS** (Elastic Kubernetes Service — Kubernetes with the control plane managed by AWS). The app is small but complete, and it maps perfectly onto the three tiers we just described:

- **Frontend** — a **React** single-page app the user opens in the browser.
- **Backend** — a **Node.js / Express** API that serves `/api/tasks` (create, list, complete, delete tasks).
- **Database** — **MongoDB**, where the tasks are stored.

The three-tier app on Amazon EKS



Here is the whole journey for this project, end to end. Don't worry about the details yet — each step gets its own chapter.



The journey: from an IAM user to a public URL on Amazon EKS.

NOTE The full [#TWSThreeTierAppChallenge](#) repo also contains Jenkins pipelines, Terraform, and Argo CD for a complete DevSecOps pipeline. This guide focuses on the **core Kubernetes + EKS deployment** — getting the three tiers live on a real cluster with a public URL. That is the heart of the project; the CI/CD wrapping is a topic for another day.

WATCH OUT EKS is **not free**. A cluster's control plane costs about \$0.10/hour, plus the EC2 worker nodes and the load balancer. Follow the **cleanup** chapter at the end the moment you're done, or you will run up a bill.

4. The Tools You'll Need

You don't need all of these before you start — the toolbox chapter walks you through installing each one on a small EC2 "control machine." This is the master list so the names aren't a surprise.

Tool	What it does
AWS CLI	Talks to your AWS account (login, ECR, EKS config)
Docker	Builds the container images for each tier and pushes them
kubectl	The command-line remote control for any Kubernetes cluster
eksctl	Creates and deletes the Amazon EKS cluster with one command
Helm	The package manager for Kubernetes — used here to install the AWS Load Balancer Controller

TIP This project comes from a **TrainWithShubham** live session. The application code and all Kubernetes manifests live in a public GitHub repository (listed in the Appendix). Cloning the repo is the fastest way to follow along — you get every manifest exactly as shown here. Look for the green **"From the repo:"** link under each heading; it points at the exact source file so you can verify everything yourself.

PART II

Deploying LIVE on Amazon EKS

Set up the toolbox, push images to ECR, build the cluster with `eksctl`, deploy all three tiers, and expose them to the internet through an ALB Ingress.

5. Setting Up the Toolbox

Watch from 0:12:14

Everything in this project is driven from one small **Ubuntu EC2 instance** — your "control machine." It is *not* part of the cluster; it's just where you run `aws`, `docker`, `kubectl`, and `eksctl`. First, create an IAM user and a `t2.medium`-ish Ubuntu EC2, SSH in, and install the tools.

Install the AWS CLI

From the repo: [TWSThreeTierAppChallenge/README.md — Step 3](#)

```
# Download the official AWS CLI v2 installer (a zip archive)
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
# Install unzip, then unpack the archive
sudo apt install unzip
unzip awscliv2.zip
# Run the installer (puts the 'aws' command in /usr/local/bin)
sudo ./aws/install -i /usr/local/aws-cli -b /usr/local/bin --update
# Enter your IAM access key, secret key, region, and output format
aws configure
```

`aws configure` is the important line: it stores the **access key** and **secret key** of the IAM user you created (give it `AdministratorAccess` for this learning project), so every later `aws` and `eksctl` command can act on your account.

Install Docker, kubectl, and eksctl

From the repo: [TWSThreeTierAppChallenge/README.md — Steps 4–6](#)

```
# --- Docker: builds and runs container images ---
sudo apt-get update
sudo apt install docker.io
# Let your user talk to the Docker daemon without sudo every time
sudo chown $USER /var/run/docker.sock

# --- kubectl: the Kubernetes command-line tool ---
curl -o kubectl
https://amazon-eks.s3.us-west-2.amazonaws.com/1.19.6/2021-01-05/bin/linux/amd64/kubectl
chmod +x ./kubectl # make it executable
sudo mv ./kubectl /usr/local/bin # put it on your PATH
kubectl version --short --client # confirm it runs

# --- eksctl: one-command EKS cluster creation ---
curl --silent --location
"https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname
-s)_amd64.tar.gz" | tar xz -C /tmp
sudo mv /tmp/eksctl /usr/local/bin
eksctl version # confirm it runs
```

TIP `kubectl` and `eksctl` are just single binaries. The pattern is always the same: download the file, `chmod +x` to make it runnable, and `mv` it into `/usr/local/bin` so you can call it from anywhere. If a newer `kubectl` version exists, grab that — the URL above pins an older one.

6. Containerizing the App and Pushing to ECR

Watch from 0:31:28

Kubernetes runs **container images**, not source code. So before we can deploy, each tier's code must be built into an image and pushed to a registry the cluster can pull from. Here we use **Amazon ECR** (Elastic Container Registry — AWS's private Docker Hub).

The repo already contains a `Dockerfile` for each tier. You build them like this:

From the repo: [TWSThreeTierAppChallenge/Application-Code/.../Dockerfile](#)

```
# Build the backend image from the Dockerfile in the current folder
docker build -t backend .
# Build the frontend image (run this inside the frontend/ folder)
docker build -t frontend .
```

Then create one ECR repository per image and push. AWS shows you the exact push commands in the ECR console ("View push commands"), but they follow this shape:

```
# 1) Log Docker in to your ECR registry (region and account id are yours)
aws ecr get-login-password --region us-east-1 \
  | docker login --username AWS --password-stdin <acct>.dkr.ecr.us-east-1.amazonaws.com
# 2) Tag your local image with the full ECR repository URL
docker tag backend:latest <acct>.dkr.ecr.us-east-1.amazonaws.com/backend:latest
# 3) Push it up to ECR so the cluster can pull it later
docker push <acct>.dkr.ecr.us-east-1.amazonaws.com/backend:latest
```

WATCH OUT The image name **must** be tagged with the full ECR URL (`<acct>.dkr.ecr.<region>.amazonaws.com/backend:latest`) before you push, or Docker won't know where to send it. This tag is exactly the value that later goes into the Deployment's `image` field.

Because ECR is a **private** registry, the cluster needs a credential to pull from it. That credential is a Kubernetes Secret named `ecr-registry-secret` (you'll see the Deployments reference it under `imagePullSecrets`). Create it once, after the cluster exists:

```
# Create a docker-registry pull secret so pods can pull private ECR images
kubectl create secret docker-registry ecr-registry-secret \
  --docker-server=<acct>.dkr.ecr.us-east-1.amazonaws.com \
  --docker-username=AWS \
  --docker-password=$(aws ecr get-login-password --region us-east-1) \
  -n three-tier
```

7. Creating the EKS Cluster

Watch from 1:02:37

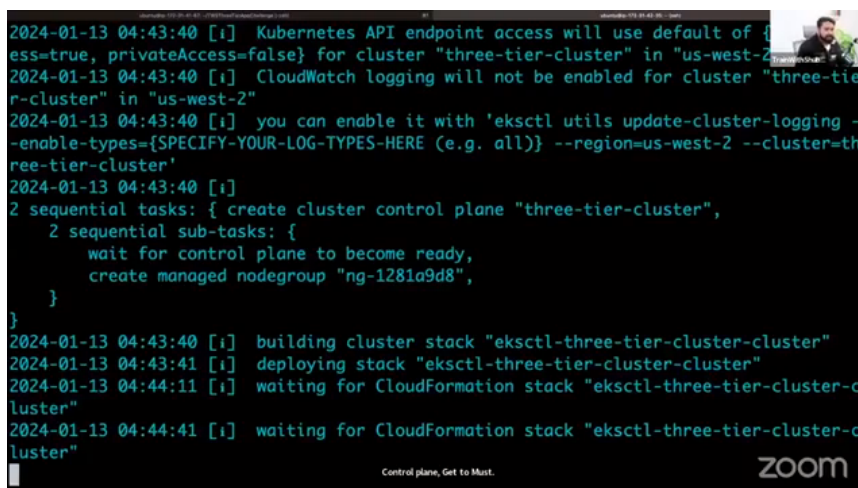
Now the star of the show. **eksctl** builds an entire EKS cluster — control plane, networking, and worker nodes — from a single command.

From the repo: [TWSThreeTierAppChallenge/README.md — Step 7](#)

```
# Create an EKS cluster named 'three-tier-cluster' with 2 t2.medium worker nodes
eksctl create cluster --name three-tier-cluster --region us-west-2 --node-type t2.medium
--nodes-min 2 --nodes-max 2

# Point kubectl at the new cluster (writes ~/.kube/config)
aws eks update-kubeconfig --region us-west-2 --name three-tier-cluster

# Confirm the worker nodes are Ready
kubectl get nodes
```

A terminal window showing the output of the `eksctl create cluster` command. The output indicates that the cluster 'three-tier-cluster' is being created in the 'us-west-2' region with 't2.medium' worker nodes. It shows the creation of the control plane and the managed node group. The terminal also displays the status of the cluster stack and the worker node group, indicating that they are being deployed and waiting for CloudFormation to complete. A Zoom logo is visible in the bottom right corner of the terminal window.

```
2024-01-13 04:43:40 [i] Kubernetes API endpoint access will use default of {
2024-01-13 04:43:40 [i] CloudWatch logging will not be enabled for cluster "three-tier-cluster" in "us-west-2"
2024-01-13 04:43:40 [i] you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --region=us-west-2 --cluster=three-tier-cluster'
2024-01-13 04:43:40 [i]
2 sequential tasks: { create cluster control plane "three-tier-cluster",
  2 sequential sub-tasks: {
    wait for control plane to become ready,
    create managed nodegroup "ng-1281a9d8",
  }
}
2024-01-13 04:43:40 [i] building cluster stack "eksctl-three-tier-cluster-cluster"
2024-01-13 04:43:41 [i] deploying stack "eksctl-three-tier-cluster-cluster"
2024-01-13 04:44:11 [i] waiting for CloudFormation stack "eksctl-three-tier-cluster-cluster"
2024-01-13 04:44:41 [i] waiting for CloudFormation stack "eksctl-three-tier-cluster-cluster"
Control plane, Get to Must. zoom
```

`eksctl` builds the cluster by creating AWS CloudFormation stacks — the control plane first, then the worker node group. This takes about 15 minutes.

That first command does a *lot*: it provisions the managed control plane, a VPC, and a node group of two EC2 workers. It takes roughly **15 minutes** — a perfect coffee break. When `kubectl get nodes` shows two nodes in **STATUS: Ready**, your cluster is live.

ANALOGY With a hand-built cluster (using a tool like [kubeadm](#)) you assemble everything brick by brick — control plane, networking, node health, all by hand. With **eksctl**, you hand AWS the blueprint and it builds the house — control plane and all — while you wait. That's the trade: EKS costs money but saves you the hardest, most error-prone work.

INTERVIEW What does `aws eks update-kubeconfig` do? It writes the new cluster's connection details (endpoint + auth) into your local `~/.kube/config` so `kubectl` knows which cluster to talk to and how to authenticate. Without it, `kubectl` has no idea your EKS cluster exists.

8. Deploying the Three Tiers

Watch from 1:11:39

The manifests live in the repo's `Kubernetes-Manifests-file/` folder, split by tier: `Database/`, `Backend/`, and `Frontend/`. Every object is placed in a namespace called `three-tier`, so create that first.

```
# A namespace keeps this project's objects tidy and separate
kubectl create namespace three-tier
```

WATCH OUT Every manifest in this repo hard-codes `namespace: three-tier`. The namespace you *create* must match the namespace *inside the manifests*, or `kubectl apply` fails with "namespace not found." (The live session created a namespace called `workshop`; if you copy that, you must also change the `namespace:` field in every manifest. Simplest is to stick with `three-tier` everywhere.)

The Data tier: Secret, storage, then MongoDB

Watch from 1:18:00

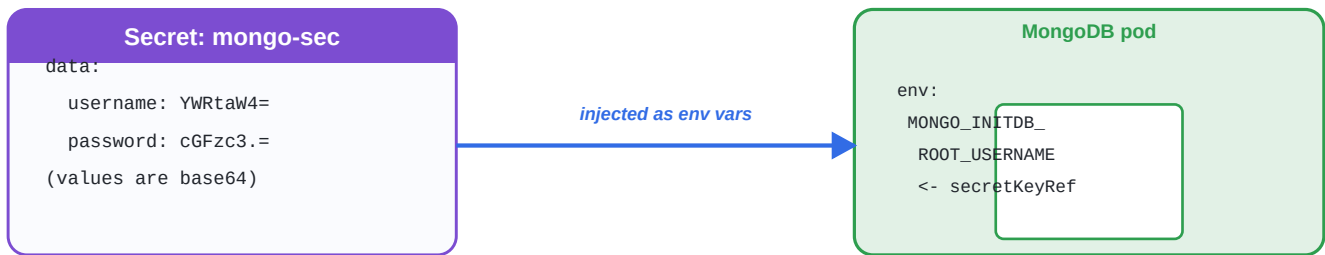
A database needs three things before it can run: a **password** (kept in a Secret), **durable storage** (a PV + PVC), and the **MongoDB Deployment + Service** itself.

First, the Secret. It holds the Mongo username and password — but notice the values look scrambled.

From the repo: [Kubernetes-Manifests-file/Database/secrets.yaml](#)

```
apiVersion: v1
kind: Secret
metadata:
  namespace: three-tier
  name: mongo-sec
type: Opaque
data:
  password: cGFzc3dvcmQxMjM= # "password123" encoded in base64
  username: YWRtaW4= # "admin" encoded in base64
```

The values are **base64-encoded**, not encrypted — base64 is just a reversible text encoding so binary/awkward characters travel safely inside YAML. A Secret keeps these values out of your Deployment files; the database Pod reads them as environment variables.



A Secret keeps passwords out of your manifests; Kubernetes feeds them into the pod as environment variables.

NOTE To create your own values: `echo -n 'admin' | base64` gives `YWRtaW4=`. To read one back: `echo 'YWRtaW4=' | base64 -d`. Because base64 is *not* encryption, never commit real production passwords to Git — use a sealed-secrets tool or a cloud secret manager for that.

Next, the storage. A **PersistentVolume** is a slice of real disk; a **PersistentVolumeClaim** is the Pod's request to use it.

From the repo: [Kubernetes-Manifests-file/Database/pv.yaml](#)

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mongo-pv
  namespace: three-tier
spec:
  capacity:
    storage: 1Gi # size of this volume
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce # mounted read-write by a single node
  hostPath:
    path: /data/db # a folder on the worker node's own disk
```

From the repo: [Kubernetes-Manifests-file/Database/pvc.yaml](#)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongo-volume-claim
  namespace: three-tier
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: "" # "" = bind to the PV above, don't auto-provision
  resources:
    requests:
      storage: 1Gi # ask for 1Gi; Kubernetes binds it to mongo-pv
```

Now MongoDB itself. The Deployment mounts the PVC for its data directory and reads the username/password from the Secret.

From the repo: [Kubernetes-Manifests-file/Database/deployment.yaml](#)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  namespace: three-tier
  name: mongodb
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongodb
  template:
    metadata:
      labels:
        app: mongodb
    spec:
      containers:
        - name: mon
          image: mongo:4.4.6
          env:
            - name: MONGO_INITDB_ROOT_USERNAME
              valueFrom:
                secretKeyRef: # read this value from the Secret...
                name: mongo-sec
                key: username # ...specifically its "username" key
            - name: MONGO_INITDB_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mongo-sec
                  key: password
          ports:
            - containerPort: 27017 # MongoDB's standard port
          volumeMounts:
            - name: mongo-volume
              mountPath: /data/db # where MongoDB writes its files
      volumes:
        - name: mongo-volume
          persistentVolumeClaim:
            claimName: mongo-volume-claim # back the volume with our PVC

```

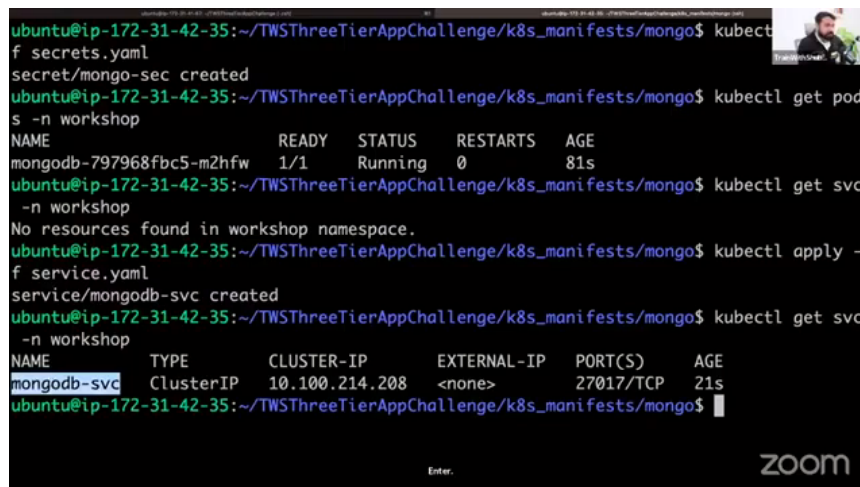
And the Service that gives MongoDB a stable in-cluster name, `mongodb-svc`:

From the repo: [Kubernetes-Manifests-file/Database/service.yaml](#)

```

apiVersion: v1
kind: Service
metadata:
  namespace: three-tier
  name: mongodb-svc
spec:
  selector:
    app: mongodb # send traffic to the mongodb pod
  ports:
    - name: mongodb-svc
      protocol: TCP
      port: 27017
      targetPort: 27017
# No 'type' field -> defaults to ClusterIP -> internal only. Perfect for a database.

```



```

ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ kubectl
f secrets.yaml
secret/mongo-sec created
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ kubectl get pod
s -n workshop
NAME                                READY   STATUS    RESTARTS   AGE
mongodb-797968fbc5-m2hfw            1/1     Running   0           81s
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ kubectl get svc
-n workshop
No resources found in workshop namespace.
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ kubectl apply -
f service.yaml
service/mongodb-svc created
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$ kubectl get svc
-n workshop
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
mongodb-svc ClusterIP   10.100.214.208 <none>        27017/TCP   21s
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests/mongo$

```

The database tier live: the mongodb Pod is Running, and `kubectl get svc` shows mongodb-svc with a ClusterIP — an internal-only address the backend will use.

The Application tier: the Node.js API

Watch from 1:31:54

The backend runs your ECR image, connects to MongoDB **by service name**, and — importantly — declares health checks so Kubernetes knows when it's actually ready.

From the repo: [Kubernetes-Manifests-file/Backend/deployment.yaml](#)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
  namespace: three-tier
  labels:
    role: api
spec:
  replicas: 2 # two copies for availability
  selector:
    matchLabels:
      role: api
  template:
    metadata:
      labels:

```

```

    role: api
spec:
  imagePullSecrets:
  - name: ecr-registry-secret # credential to pull the private ECR image
  containers:
  - name: api
    image: <acct>.dkr.ecr.us-east-1.amazonaws.com/backend:latest # your ECR image
    imagePullPolicy: Always
    env:
      - name: MONGO_CONN_STR
        value: mongodb://mongodb-svc:27017/todo?directConnection=true # DB by service
name
      - name: MONGO_USERNAME
        valueFrom:
          secretKeyRef:
            name: mongo-sec
            key: username
      - name: MONGO_PASSWORD
        valueFrom:
          secretKeyRef:
            name: mongo-sec
            key: password
    ports:
      - containerPort: 3500 # the port the API listens on
    livenessProbe: # is the container alive? restart if not
      httpGet:
        path: /ok
        port: 3500
      initialDelaySeconds: 2
      periodSeconds: 5
    readinessProbe: # is it ready for traffic yet? hold traffic until yes
      httpGet:
        path: /ok
        port: 3500
      initialDelaySeconds: 5
      periodSeconds: 5

```

The single most important line is `MONGO_CONN_STR: mongodb://mongodb-svc:27017/todo`. The host is `mongodb-svc` — the Service name from the previous chapter — not an IP. That's service discovery doing its job: the API finds the database by name, and it keeps working even when the Mongo Pod is recreated with a new IP.

KEY IDEA **Liveness** and **readiness probes** are how Kubernetes tells "running" from "actually working." The liveness probe restarts a hung container; the readiness probe keeps traffic away from a Pod until it can serve. Both hit `/ok` on port 3500 here — a tiny health endpoint the app exposes.

The backend Service is internal (`ClusterIP`) — only the frontend and Ingress need to reach it:

From the repo: [Kubernetes-Manifests-file/Backend/service.yaml](#)

```

apiVersion: v1
kind: Service
metadata:
  name: api
  namespace: three-tier
spec:
  ports:
    - port: 3500
      protocol: TCP
  type: ClusterIP # internal address for the API
  selector:
    role: api # target the api pods

```

The Presentation tier: the React frontend

Watch from 1:37:48

The frontend Deployment is the same shape, running the React image and told where the backend lives via `REACT_APP_BACKEND_URL`:

From the repo: [Kubernetes-Manifests-file/Frontend/deployment.yaml](#)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  namespace: three-tier
  labels:
    role: frontend
spec:
  replicas: 1
  selector:
    matchLabels:
      role: frontend
  template:
    metadata:
      labels:
        role: frontend
    spec:
      imagePullSecrets:
        - name: ecr-registry-secret
      containers:
        - name: frontend
          image: <acct>.dkr.ecr.us-east-1.amazonaws.com/frontend:latest # your ECR image
          imagePullPolicy: Always
          env:
            - name: REACT_APP_BACKEND_URL
              value: "http://backend.example.com/api/tasks" # public URL the browser calls
          ports:
            - containerPort: 3000

```

NOTE `REACT_APP_BACKEND_URL` must be a URL the **browser** can reach (it runs on the user's machine, outside the cluster), which is why it points at the public Ingress host and the `/api` path — not at the internal `api` Service. Set it to your own Ingress domain. The repo ships an example domain here; swap in yours.

Its Service is also `ClusterIP`; the Ingress will expose it publicly. Now apply everything. From inside the manifests folder:

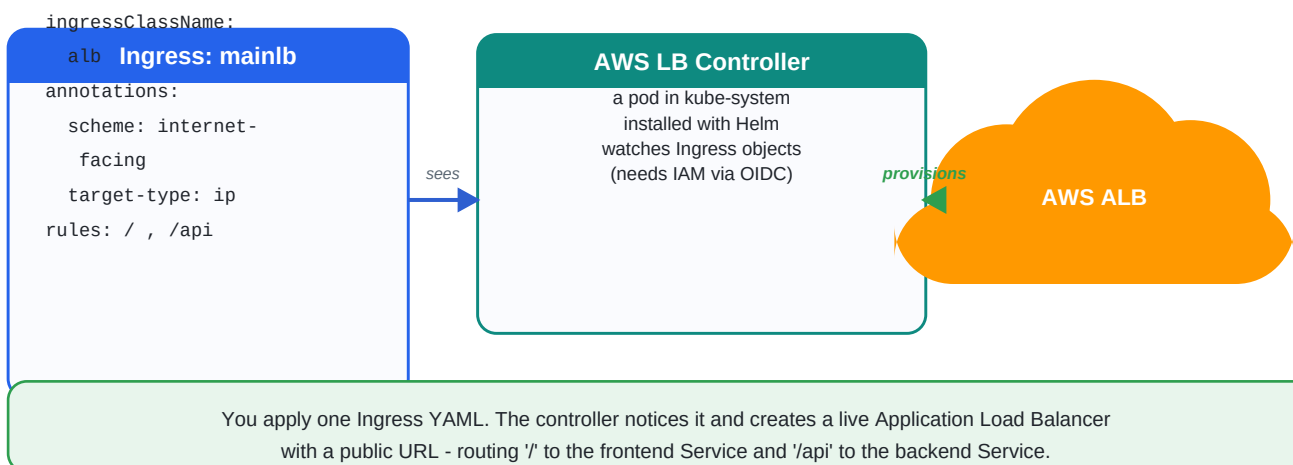
```
# Apply every manifest in the current folder (and subfolders won't be included -  
# apply each tier's folder, or use -R for recursive)  
kubectl apply -f .  
# Watch all three tiers come up  
kubectl get pods -n three-tier
```

9. Exposing the App with an ALB Ingress

Watch from 1:49:23

The Pods are running, but nothing outside the cluster can reach them yet — all three Services are `ClusterIP` (internal). To let the world in, we use an **Ingress**: a rule that routes web traffic by host and path into Services. On EKS, an Ingress is fulfilled by a real **AWS Application Load Balancer (ALB)** — but only once you install the controller that knows how to create one.

How an Ingress becomes a real AWS load balancer



Install the AWS Load Balancer Controller

Watch from 1:50:15

This controller runs inside the cluster, watches for Ingress objects, and provisions a matching ALB in your AWS account. Installing it takes three moving parts: an **IAM policy** (permissions), an **IAM service account** tied to the cluster via **OIDC** (so the controller can act on AWS), and the controller itself (installed with **Helm**).

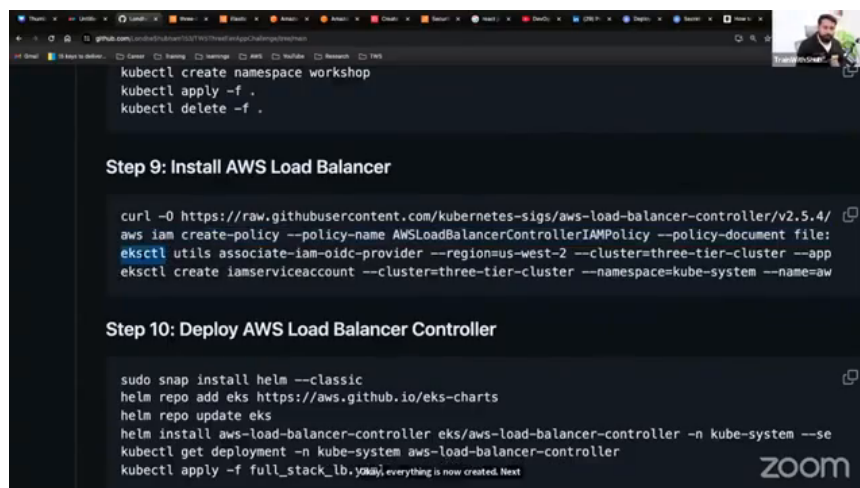
From the repo: [TWSThreeTierAppChallenge/README.md](#) — [Steps 9–10](#)

```
# 1) Download the IAM policy the controller needs, and create it in your account
curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.5.4/docs/install/iam_policy.json
aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy --policy-document file://iam_policy.json
```

```
# 2) Let the cluster use IAM roles for service accounts (enable the OIDC provider)
eksctl utils associate-iam-oidc-provider --region=us-west-2 --cluster=three-tier-cluster --approve
```

```
# 3) Create a Kubernetes service account bound to that IAM policy
eksctl create iamserviceaccount --cluster=three-tier-cluster --namespace=kube-system \
  --name=aws-load-balancer-controller \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --attach-policy-arn=arn:aws:iam::<acct>:policy/AWSLoadBalancerControllerIAMPolicy \
  --approve --region=us-west-2
```

```
# 4) Install the controller itself with Helm
sudo snap install helm --classic
helm repo add eks https://aws.github.io/eks-charts
helm repo update eks
helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system \
  --set clusterName=three-tier-cluster \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller
# Confirm the controller Deployment is up
kubectl get deployment -n kube-system aws-load-balancer-controller
```



Installing the AWS Load Balancer Controller with Helm, following the repo's Step 9 and Step 10.

WATCH OUT In the repo the Helm command shows `--set clusterName=my-cluster`. That's a placeholder — set it to your **real** cluster name (`three-tier-cluster`). If the controller is told the wrong cluster name, it can't find your cluster and no ALB will ever appear. Also replace `<acct>` with your 12-digit AWS account ID in the policy ARN.

Apply the Ingress

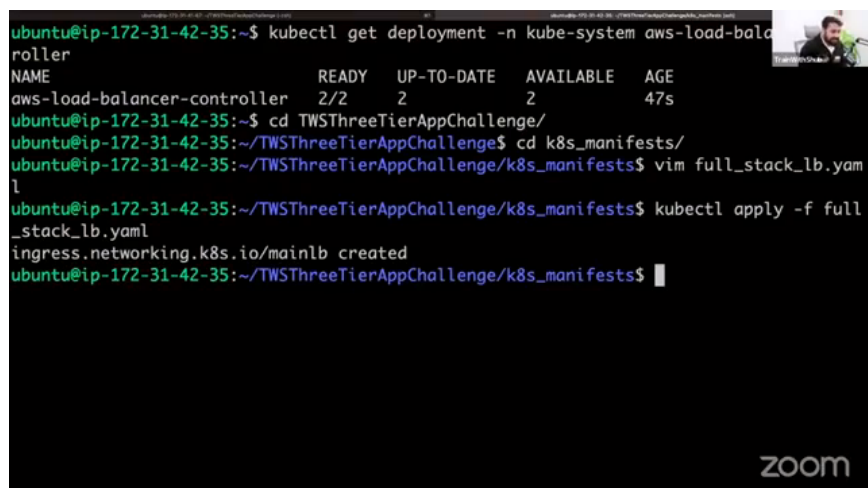
Watch from 2:13:35

With the controller running, apply the Ingress. Its annotations are what turn it into a real ALB.

From the repo: [Kubernetes-Manifests-file/ingress.yaml](#)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mainlb
  namespace: three-tier
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing # a public ALB
    alb.ingress.kubernetes.io/target-type: ip # send traffic straight to pod IPs
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP": 80}]'
spec:
  ingressClassName: alb # use the AWS Load Balancer Controller
  rules:
    - host: backend.example.com # your domain
      http:
        paths:
          - path: /api # /api/... goes to the backend API
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 3500
          - path: / # everything else goes to the React frontend
            pathType: Prefix
            backend:
              service:
                name: frontend
                port:
                  number: 3000
```

```
# Create the Ingress -> the controller now provisions an AWS ALB
kubectl apply -f ingress.yaml
# Watch for the ADDRESS column to fill in with the ALB's DNS name (takes a few minutes)
kubectl get ingress -n three-tier
```



```
ubuntu@ip-172-31-42-35:~$ kubectl get deployment -n kube-system aws-load-balancer-controller
NAME                                READY  UP-TO-DATE  AVAILABLE  AGE
aws-load-balancer-controller        2/2    2            2           47s
ubuntu@ip-172-31-42-35:~$ cd TWSThreeTierAppChallenge/
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge$ cd k8s_manifests/
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ vim full_stack_lb.yaml
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$ kubectl apply -f full_stack_lb.yaml
ingress.networking.k8s.io/mainlb created
ubuntu@ip-172-31-42-35:~/TWSThreeTierAppChallenge/k8s_manifests$
```

Applying the Ingress: [ingress.networking.k8s.io/mainlb](#) created. Within a few minutes the AWS Load Balancer Controller provisions a real ALB and fills in the ingress ADDRESS.

Read the two rules top to bottom: any request whose path starts with `/api` is sent to the `api` Service on port 3500; **everything else** (`/`) goes to the `frontend` Service on port 3000. One load balancer, two tiers, split by path. When `kubectl get ingress` shows an `ADDRESS` (a long `...elb.amazonaws.com` name), open it in your browser — the to-do app is live on the internet.

INTERVIEW *Ingress vs a LoadBalancer Service — what's the difference?* A `LoadBalancer` Service gives **one** Service its own cloud load balancer. An **Ingress** puts **one** load balancer in front of **many** Services and routes by host/path — so a single ALB can serve `/` to the frontend and `/api` to the backend. It's the cost-effective, production-friendly way to expose multiple tiers.

10. Cleanup — Don't Skip This

Watch from 2:33:27

EKS bills by the hour. When you're done exploring, tear the whole thing down:

```
# Delete the cluster, its node group, and networking in one command
eksctl delete cluster --name three-tier-cluster --region us-west-2
```

Then, in the AWS console, double-check that the **load balancer** created by the Ingress is gone, delete any leftover **security groups** from the steps above, and **stop or terminate** the toolbox EC2 instance from the start of this project.

KEY IDEA `eksctl` giveth and `eksctl` taketh away. Because it created the cluster via CloudFormation, `eksctl delete cluster` removes all of it cleanly. The two things it can't always clean up are the **ALB** (created by the controller, not `eksctl`) and your **toolbox EC2** — check those by hand so nothing keeps billing.

APPENDICES

Appendices

A command cheat sheet, the source video & repository, interview questions, and a glossary.

11. Command Cheat Sheet

Every command you need for this project, grouped by tool. Keep this page handy.

kubectl — the everyday driver

```
# --- looking around ---
kubectl get nodes # list cluster nodes and their status
kubectl get pods -n three-tier # pods in the project namespace
kubectl get all -n three-tier # every object in the namespace
kubectl get svc,ingress -n three-tier # services and the ingress (watch for the ALB address)
kubectl get pods -o wide # add the NODE column (which node runs each pod)

# --- creating & changing ---
kubectl create namespace three-tier # make the namespace
kubectl apply -f . # apply every manifest in this folder
kubectl apply -f ingress.yaml # apply a single manifest
kubectl scale deployment api --replicas=3 # change the number of API pods

# --- debugging (your best friends) ---
kubectl describe pod <pod> -n three-tier # events + config; read the bottom for "why"
kubectl logs <pod> -n three-tier # the container's own output
kubectl logs -f <pod> -n three-tier # follow logs live
kubectl exec -it <pod> -n three-tier -- bash # open a shell inside a running pod
kubectl get events -n three-tier --sort-by=.lastTimestamp # recent events

# --- context & cleanup ---
kubectl config set-context --current --namespace three-tier # skip -n every time
kubectl delete -f . # tear down what these manifests created
```

eksctl — create and delete the EKS cluster

```
# Create a cluster with a managed node group (~15 minutes)
eksctl create cluster --name three-tier-cluster --region us-west-2 --node-type t2.medium
--nodes-min 2 --nodes-max 2

# Enable IAM Roles for Service Accounts (needed by the LB controller)
eksctl utils associate-iam-oidc-provider --region=us-west-2 --cluster=three-tier-cluster
--approve

# Create the IAM service account for the AWS Load Balancer Controller
eksctl create iamserviceaccount --cluster=three-tier-cluster --namespace=kube-system \
--name=aws-load-balancer-controller --role-name AmazonEKSLoadBalancerControllerRole \
--attach-policy-arn=arn:aws:iam::<acct>:policy/AWSLoadBalancerControllerIAMPolicy
--approve --region=us-west-2

# Point kubectl at the cluster
aws eks update-kubeconfig --region us-west-2 --name three-tier-cluster

# Delete the whole cluster when you're done (stops the billing)
eksctl delete cluster --name three-tier-cluster --region us-west-2
```

docker & ECR — build and store the images

```

docker build -t backend . # build an image from a Dockerfile
docker tag backend:latest <acct>.dkr.ecr.us-east-1.amazonaws.com/backend:latest # tag for ECR
docker push <acct>.dkr.ecr.us-east-1.amazonaws.com/backend:latest # push to ECR
# Log Docker in to a private ECR registry
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin <acct>.dkr.ecr.us-east-1.amazonaws.com
# Create the pull secret so pods can pull the private images
kubectl create secret docker-registry ecr-registry-secret \
  --docker-server=<acct>.dkr.ecr.us-east-1.amazonaws.com --docker-username=AWS \
  --docker-password=$(aws ecr get-login-password --region us-east-1) -n three-tier

```

helm — install the AWS Load Balancer Controller

```

sudo snap install helm --classic # install Helm itself
helm repo add eks https://aws.github.io/eks-charts
helm repo update eks
helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system \
  --set clusterName=three-tier-cluster \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller

```

12. Important Links and Repositories

The source video (TrainWithShubham):

- [Deploying a Three-tier Application LIVE on AWS EKS](#) — the full live session this guide is based on.
- Channel & courses: trainwithshubham.com

The project repository (source of truth for all code):

- Three-tier app on EKS: github.com/LondheShubham153/TWSThreeTierAppChallenge
- Kubernetes manifests: [Kubernetes-Manifests-file/](#) (Database / Backend / Frontend + [ingress.yaml](#))
- Application code & Dockerfiles: [Application-Code/](#)

Official documentation:

- Amazon EKS: docs.aws.amazon.com/eks · eksctl: eksctl.io
- AWS Load Balancer Controller: kubernetes-sigs.github.io/aws-load-balancer-controller
- Amazon ECR: docs.aws.amazon.com/ecr
- Kubernetes: kubernetes.io/docs · Helm: helm.sh/docs

13. Interview Questions and Answers

A round-up of the questions this project prepares you for.

Q. What is a three-tier architecture?

A. A split of an app into a **presentation** tier (the frontend/UI), an **application** tier (the backend/API and business logic), and a **data** tier (the database). Each can be scaled, updated, and secured independently. On Kubernetes each tier is a Deployment plus a Service.

Q. What's the difference between a Pod, a Deployment, and a Service?

A. A **Pod** is the smallest deployable unit (one or more containers). A **Deployment** keeps a desired number of identical Pods running and heals/scales them. A **Service** gives those Pods one stable address and load-balances across them. You usually create a Deployment + Service pair per tier.

Q. ClusterIP vs NodePort vs LoadBalancer vs Ingress?

A. **ClusterIP** is internal-only (the default) — used here for all three tiers. **NodePort** opens a fixed port on every node's IP. **LoadBalancer** provisions one cloud load balancer per Service. **Ingress** puts one load balancer in front of many Services and routes by host/path — the production choice, and how this project exposes `/` (frontend) and `/api` (backend) through a single ALB.

Q. What is Amazon EKS, and what does AWS manage for you?

A. EKS is **managed Kubernetes**: AWS runs and scales the **control plane** (API server, etcd, scheduler) for you, so you only manage the worker nodes and your apps. You trade a per-hour cost for skipping the hardest, most error-prone part of running Kubernetes.

Q. What does `aws eks update-kubeconfig` do?

A. It writes the cluster's connection details (endpoint + auth) into your local `~/.kube/config` so `kubectl` knows which cluster to talk to and how to authenticate. Without it, `kubectl` has no idea your EKS cluster exists.

Q. How do two tiers find each other in a cluster?

A. Through **service discovery**: a Service's name (e.g. `mongodb-svc`) resolves via cluster DNS to a healthy Pod behind it. You configure the backend with the *service name*, never a Pod IP, so it keeps working as Pods are recreated. Here the API reaches the database at `mongodb://mongodb-svc:27017`.

Q. Why does the app use ECR, and what is `imagePullSecrets` for?

A. ECR is AWS's **private** image registry. Because it's private, the cluster needs a credential to pull from it — a docker-registry Secret (`ecr-registry-secret`) that each Deployment references under `imagePullSecrets`.

Q. Is a Kubernetes Secret secure?

A. By default it's only **base64-encoded**, not encrypted — anyone who can read it can decode it. Keep Secrets out of Git, enable encryption-at-rest, and consider Sealed Secrets or an external secrets manager for production.

Q. PV vs PVC?

A. A **PersistentVolume** is the actual storage (supply). A **PersistentVolumeClaim** is a Pod's request for storage (demand). Pods reference the claim; Kubernetes binds it to a matching volume — separating who provides storage from who needs it. Here MongoDB's data lives on a PV so it survives Pod restarts.

Q. Liveness vs readiness probe?

A. A **liveness** probe restarts a container that has hung. A **readiness** probe keeps traffic away from a Pod until it's ready to serve (without restarting it). The backend defines both against `/ok` on port 3500, which makes rolling updates and self-healing reliable.

Q. What does the AWS Load Balancer Controller do?

A. It runs in the cluster, watches for `Ingress` objects with the `alb` class, and provisions a real AWS **Application Load Balancer** to satisfy them — routing external traffic to your Services by host and path. It needs IAM permissions, granted via an OIDC-backed service account (IRSA).

Q. How do you avoid a surprise AWS bill with EKS?

A. EKS charges for the control plane (~\$0.10/hour), the EC2 worker nodes, and the ALB. When finished, run `eksctl delete cluster`, then confirm in the console that the **ALB** and any leftover **security groups** are gone and the **toolbox EC2** is stopped — `eksctl` doesn't always remove the controller-created ALB.

14. Glossary

Term	Meaning
Cluster	A set of machines (nodes) Kubernetes runs your apps across.
Node	One machine in the cluster — on EKS, an EC2 worker instance.

Term	Meaning
Pod	The smallest deployable unit; wraps one or more containers sharing network and storage.
Deployment	Keeps a desired number of identical Pods running; heals and scales them.
Service	A stable network address and load balancer for a set of Pods.
ClusterIP	The default Service type — reachable only inside the cluster (used by all three tiers here).
Namespace	A virtual folder that isolates a group of objects; this project uses three-tier .
Manifest	A YAML file describing a desired Kubernetes object.
Secret / ConfigMap	Store sensitive / non-sensitive configuration outside the image.
PV / PVC	Persistent storage, and a Pod's request to use it.
Ingress	A rule that routes external HTTP traffic to Services by host/path.
Service discovery	Finding another tier by its Service name (via cluster DNS) instead of a Pod IP.
EKS	Amazon's managed Kubernetes — AWS runs the control plane.
eksctl	The official CLI that creates and deletes EKS clusters (via CloudFormation).
ECR	Amazon Elastic Container Registry — a private registry the cluster pulls images from.
ALB	AWS Application Load Balancer — the load balancer an EKS Ingress provisions.
AWS Load Balancer Controller	The in-cluster controller that turns Ingress objects into real ALBs.
OIDC / IRSA	IAM Roles for Service Accounts — lets an in-cluster service account assume an AWS IAM role.
Helm	The package manager for Kubernetes; installs the LB controller here.

TIP You've now deployed a real three-tier application, live on Amazon EKS, exposed to the internet through an ALB Ingress. The fastest way to make it stick is to do it again from scratch **without looking** — then break something on purpose (delete the Mongo Pod, point the API at the wrong service name) and fix it. That loop, more than any reading, is what turns these pages into a skill.

15. Video Chapter Index

The full walkthrough is one continuous **2h 35m** live session: [Deploying a Three-tier Application LIVE on AWS EKS](#). The video has no built-in chapters, so the timestamps below — and the red **"Watch from ..."** buttons under each heading throughout this guide — were mapped to the session by topic. **Click any timestamp to jump straight to that moment** in the video.

Timestamp	What's happening in the video (guide chapter)
0:00:22	Intro: the app and the plan — React + Node.js + MongoDB (Ch. 3)
0:02:14	What a three-tier app is: presentation / application / data (Ch. 1)
0:12:14	Setting up the toolbox: the EC2 control machine (Ch. 5)
0:31:28	Containerizing: docker build the tier images (Ch. 6)
0:44:40	Pushing the images to Amazon ECR (Ch. 6)

Timestamp	What's happening in the video (guide chapter)
0:58:31	Installing kubectl and eksctl (Ch. 5)
1:02:37	Creating the EKS cluster with eksctl (Ch. 7)
1:11:39	Deploying the tiers: the Kubernetes manifests (Ch. 8)
1:18:00	Data tier: Secret, PV/PVC, then MongoDB (Ch. 8)
1:31:54	Application tier: the Node.js API (Ch. 8)
1:37:48	Presentation tier: the React frontend (Ch. 8)
1:49:23	Exposing the app: why you need an Ingress controller (Ch. 9)
1:50:15	Installing the AWS Load Balancer Controller with Helm (Ch. 9)
2:13:35	Applying the Ingress to the ALB is provisioned (Ch. 9)
2:24:20	The app is live: add a task, see it stored in MongoDB
2:33:27	Cleanup: delete the cluster so it stops billing (Ch. 10)

NOTE These timestamps are a close guide, not frame-exact — this was a live session that occasionally jumps around, so start a little before each mark if you don't land right on the step.